# VIPER Programming Language — Five Lexical Errors (Detailed RAG Guide)

This document is intentionally verbose so Retrieval-Augmented Generation (RAG) systems have rich context to retrieve precise answers for university exam questions.

## Illegal Character

**Meaning:** A character that is not part of the VIPER alphabet or token set.

**Why this is a lexical error:** The lexer scans one character at a time. If a character cannot begin or continue any valid token, the DFA reaches a dead state.

### *Invalid VIPER example*

```
@
fang age = 20;
```

**Why invalid:** The symbol '@' is not defined as an operator, delimiter, keyword, identifier character, or literal delimiter.

**What the lexer/DFA detects:** The deterministic finite automaton cannot produce a valid token and raises the corresponding lexical error.

### *Valid example*

```
fang age = 20;
```

**Exam / Interview explanation:** Exam tip: Lexical errors stop tokenization before parsing.

## Illegal Identifier

**Meaning:** Identifier violates ^[a-zA-Z_][a-zA-Z0-9_]*$.

**Why this is a lexical error:** Identifiers must begin with a letter or underscore. Beginning with a digit prevents creation of an ID token.

### *Invalid VIPER example*

```
fang 123age = 20;
```

**Why invalid:** The lexer first recognizes digits, so '123age' cannot be classified as an identifier.

**What the lexer/DFA detects:** The deterministic finite automaton cannot produce a valid token and raises the corresponding lexical error.

### *Valid example*

```
fang age = 20;
```

**Exam / Interview explanation:** Remember: digits may appear after the first character only.

## Unterminated String

**Meaning:** A string starts with " but never ends.

**Why this is a lexical error:** The lexer remains inside STRING state until another quote is found. Reaching end-of-line/file produces an unterminated string error.

### *Invalid VIPER example*

```
coil name = "Fahad";
```

**Why invalid:** The closing quote completes the STRING_CONST token.

**What the lexer/DFA detects:** The deterministic finite automaton cannot produce a valid token and raises the corresponding lexical error.

### *Valid example*

```
coil name = "Fahad";
```

**Exam / Interview explanation:** Parser never receives a valid string token.

## Unterminated Multiline Comment

**Meaning:** Comment opening exists but closing */ is missing.

**Why this is a lexical error:** The lexer stays in COMMENT state until */ appears. EOF triggers the lexical error.

### *Invalid VIPER example*

```
/* comment */
```

**Why invalid:** End marker closes the comment tokenization state.

**What the lexer/DFA detects:** The deterministic finite automaton cannot produce a valid token and raises the corresponding lexical error.

### *Valid example*

```
/* comment */
```

**Exam / Interview explanation:** Comments are ignored only after they are properly terminated.

## Number Too Large

**Meaning:** Numeric literal exceeds implementation limit (VIPER spec: max number length 16 digits).

**Why this is a lexical error:** Although digits are valid individually, the completed numeric token violates implementation constraints.

### *Invalid VIPER example*

```
9999
```

**Why invalid:** The lexer reports NumberTooLong/overflow before semantic analysis.

**What the lexer/DFA detects:** The deterministic finite automaton cannot produce a valid token and raises the corresponding lexical error.

### *Valid example*

```
fang n = 9999;
```

**Exam / Interview explanation:** Exact machine limit depends on implementation, but VIPER specifies 16 digits.

## Summary Table

| | Example | Rule Violated |
|---|---|---|
| | @ | The symbol '@' is not defined as an operator, delimiter, keyword, identifier character, or literal delimiter. |
| | fang 123age = 20; | The lexer first recognizes digits, so '123age' cannot be classified as an identifier. |
| | coil name = "Fahad"; | The closing quote completes the STRING_CONST token. |
| ment | /* comment */ | End marker closes the comment tokenization state. |

| | | |
|---|---|---|
| | 9999 | The lexer reports NumberTooLong/overflow before semantic analysis. |

## Lexical Error vs Syntax Error

Lexical analysis is the first compiler phase. It converts characters into tokens using regular expressions and a DFA. Invalid characters, malformed identifiers, unterminated strings/comments, and oversized numeric literals are lexical errors because valid tokens cannot be produced. Parsing begins only after successful tokenization. Syntax errors occur when valid tokens violate the Context-Free Grammar (CFG), such as a missing semicolon, incorrect statement order, or mismatched grammar production.